

Exploring Scikit-learn in Python

Python Scikit-learn lets users perform various machine learning tasks and provides a means to implement machine learning in Python. It's basically a SciPy toolkit that features various machine learning algorithms.

Scikit-learn has small standard datasets that we don't need to download from any external website. We can just import these datasets directly from Scikit-learn. Following is a list of the datasets that come with Scikit-learn:

1. Boston House Prices Dataset
2. Iris Plants Dataset
3. Diabetes Dataset
4. Digits Dataset
5. Wine Recognition Dataset
6. Breast Cancer Dataset

What Can We Achieve Using Python Scikit-Learn?

For the most part, users accomplish three primary tasks with scikit-learn:

1. **Classification** Identifying which category an object belongs to.

Application: Spam detection

2. **Regression** Predicting a continuous variable based on relevant independent variables.

Application: Stock price predictions

3. **Clustering** Automatic grouping of similar objects into different clusters.

Application: Customer segmentation

Importing Dataset in Scikit-learn

```
In [1]: import sklearn  
import pandas as pd
```

```
In [2]: from sklearn.datasets import load_iris
```

About dataset

The Iris Dataset contains four features (length and width of sepals and petals) of 50 samples of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). These measures were used to create a linear discriminant model to classify the species. The dataset is often used in data mining, classification and clustering examples and to test algorithms.



```
In [6]: iriss = load_iris()
df_iris = pd.DataFrame(iriss.data, columns=iriss.feature_names)
```

```
In [7]: df_iris.head()
```

```
Out[7]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
In [8]: df_iris.tail()
```

```
Out[8]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

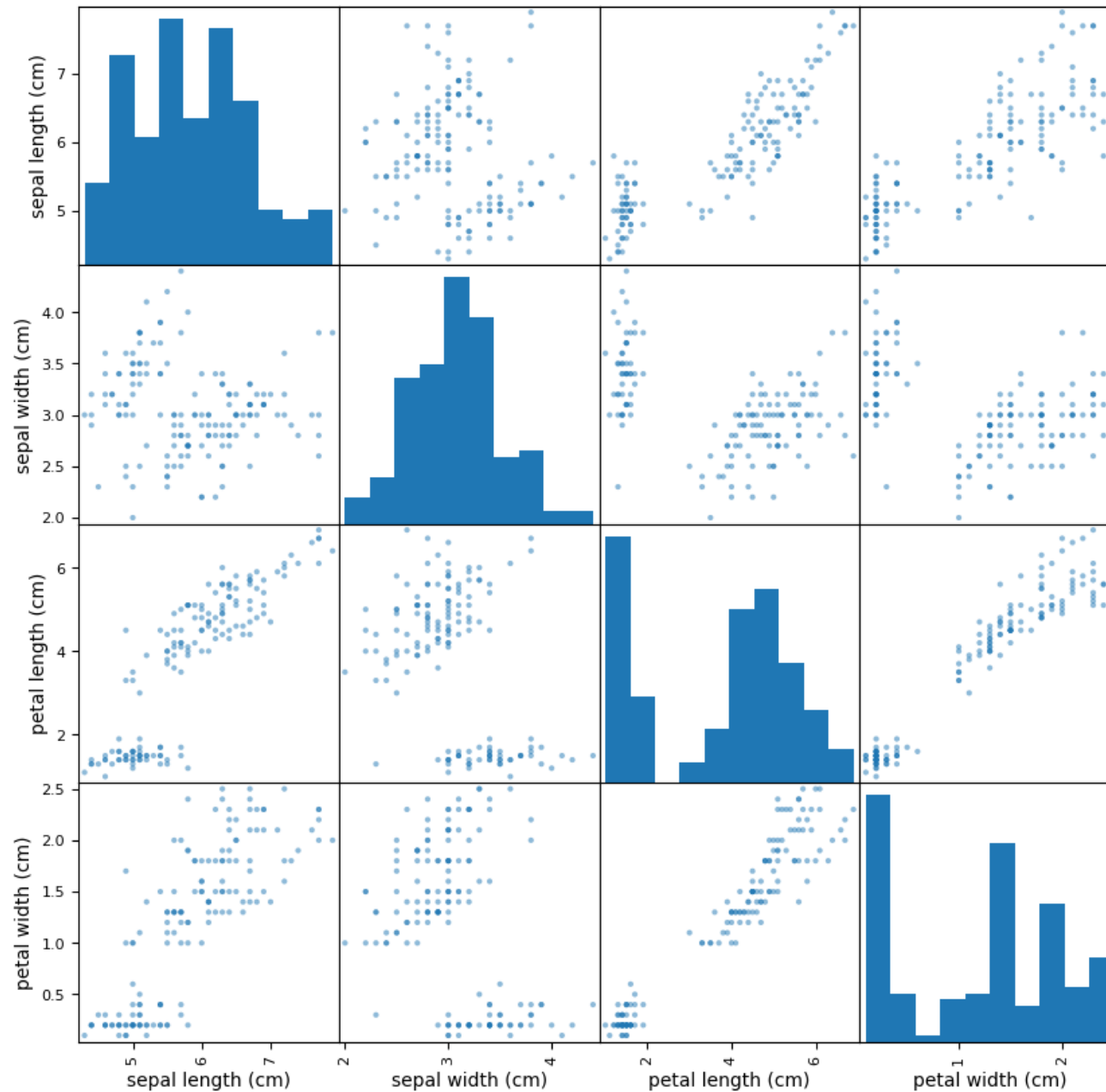
```
In [9]: df_iris.dtypes
```

```
Out[9]: sepal length (cm)    float64  
sepal width (cm)          float64  
petal length (cm)         float64  
petal width (cm)          float64  
dtype: object
```

Data Visualization Having performed data exploration with our dataset, now let's create some plots to visually represent the data in our dataset which will help us uncover more stories hidden in it.

Python has many libraries that provide functions to perform data visualizations on datasets. We can use the `.plot` extension of Pandas to create a scatterplot of features or fields of our dataset against each other, and we also need to import python matplotlib which will provide an object-oriented API to embed plots into applications.

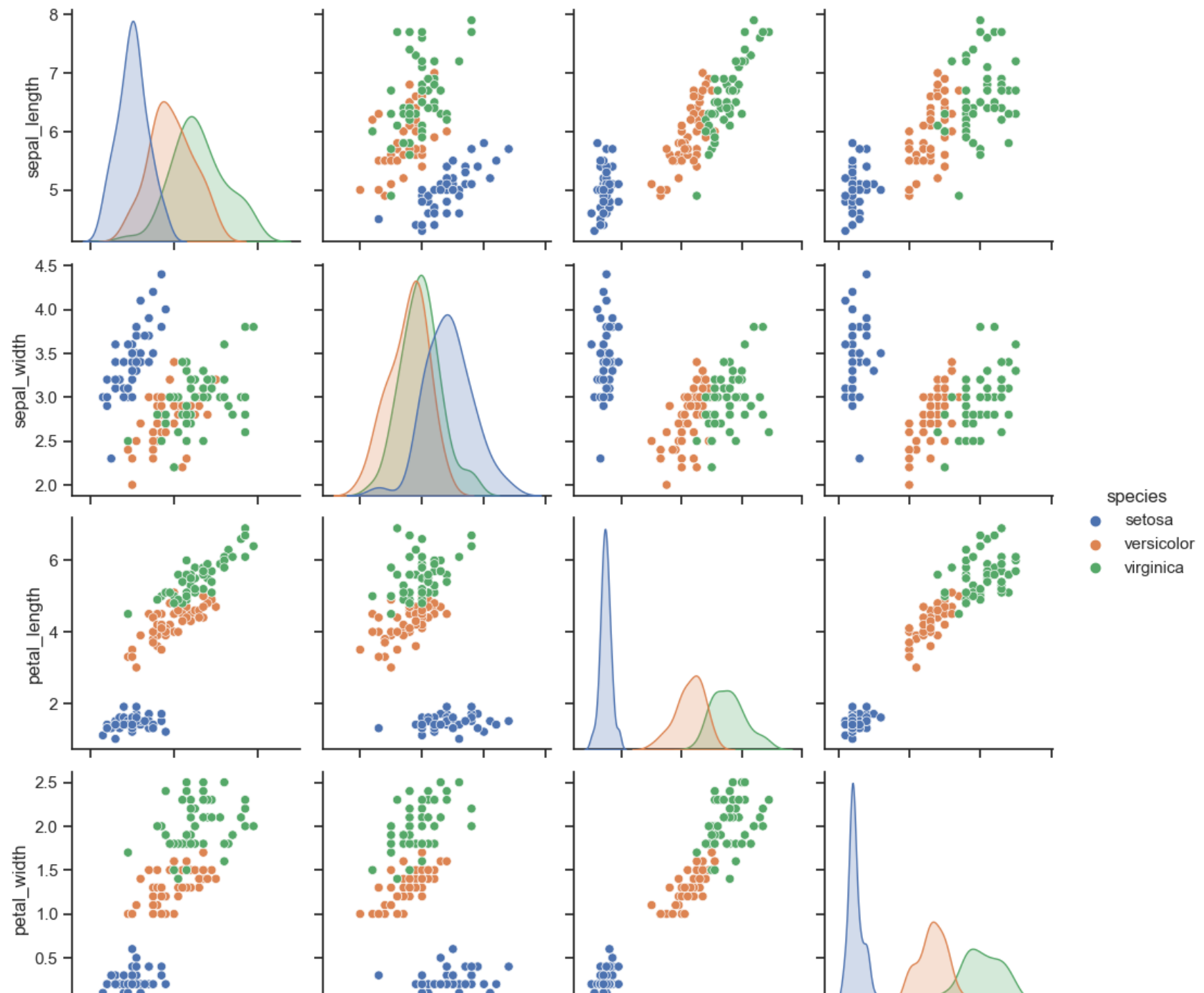
```
In [10]: from pandas.plotting import scatter_matrix  
import matplotlib.pyplot as plt  
scatter_matrix(df_iris, figsize=(10,10))  
plt.show()
```

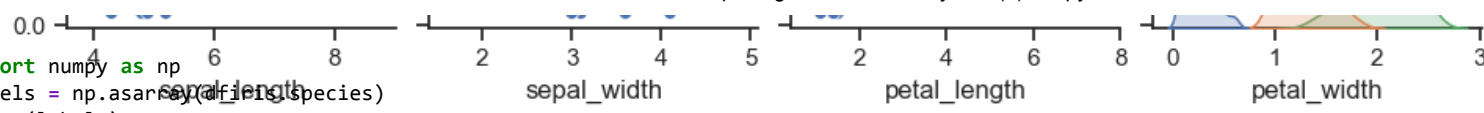


We can also use Seaborn library to create pairplots of all features in the dataset against each other. To use Seaborn, we need to import Seaborn library, first. Let's see how it is done and how to create a Seaborn pairplot.

```
In [11]: import seaborn as sns
sns.set(style="ticks", color_codes=True)
dfiris = sns.load_dataset("iris")
sns.pairplot(dfiris, hue="species")
```

```
Out[11]: <seaborn.axisgrid.PairGrid at 0x5843543910>
```



```
In [13]: import numpy as np
labels = np.asarray(['setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa', 'setosa',
'setosa', 'setosa', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'versicolor', 'versicolor', 'versicolor', 'versicolor',
'versicolor', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
'virginica', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
'virginica', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
'virginica', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
'virginica', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
'virginica', 'virginica', 'virginica', 'virginica', 'virginica', 'virginica',
'virginica', 'virginica', 'virginica'])
print(labels)
```

['setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa'
'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa'
'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa'
'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa'
'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa' 'setosa'
'setosa' 'setosa' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica' 'virginica' 'virginica' 'virginica'
'virginica' 'virginica' 'virginica']

```
In [11]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
le.fit(labels)
labels = le.transform(labels)
print(labels)
```

[0
0
1
2
2 2]

```
In [12]: from sklearn import datasets
iris=datasets.load_iris()
x=iris.data
y=iris.target
```

```
In [13]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=.5)
from sklearn import tree
```

```
In [14]: classifier=tree.DecisionTreeClassifier()
```

Now train the model using the fit method. In the fit method, pass training datasets in it. x_train and y_train are the training datasets.

```
In [15]: classifier.fit(x_train,y_train)
predictions=classifier.predict(x_test)
from sklearn.metrics import accuracy_score
print(accuracy_score(y_test,predictions)*100)
```

84.0

```
In [ ]:
```